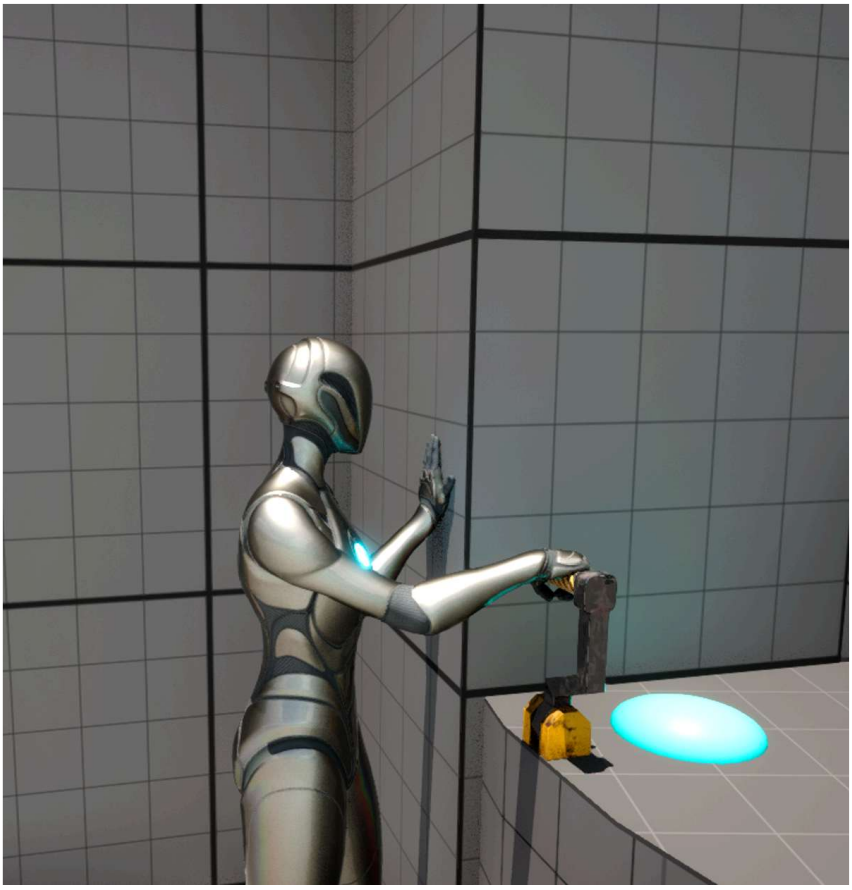
EnviroSense: Immersive Hand IK.

v1.7



Dankest Things

Contents:

What is EnviroSense: Immersive Hand IK?

EnviroSense: Immersive Hand IK was created as a modular asset that could be added to any humanoid character that would provide the ability to interact with the gameplay environment in a more immersive manner. Whilst procedural world interaction is the aim, the same system can be used to provide simple interactions with various gameplay objects, e.g. switches and levers.

EnviroSense is not a replacement for an animation driven IK enhanced specific action dedicated interaction system, but provides an accessible multipurpose solution to procedural and targeted sensing of the gameplay environment.

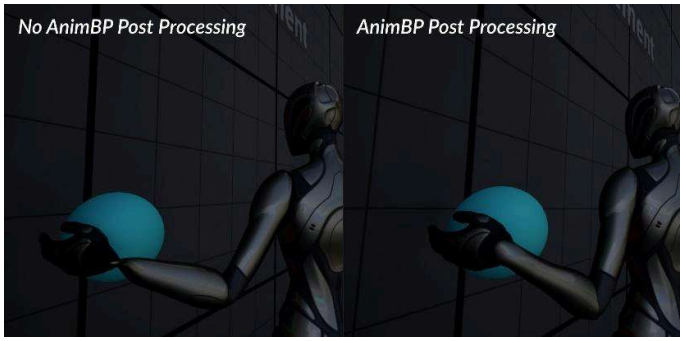
Whilst animation is replicated, this asset is not replicated in its current release.

Important Notes.

EnviroSense was developed as a tool to enhance interactions with the game world and help create an immersive experience. As added value to this, it provides a simple system for in game interactions. As mentioned, it is no replacement for specific animation-driven actions, nor can it be expected to deliver good results without consideration of game design. In short:

- do not litter your gameworld with sensory objects and allow your player to interact with them in an unregulated manner, such as running at speed, as you will get unnatural movements that will not be immersive.
- Do, consider placement of sensory objects in the context of gameplay, and how sensing should be restricited, eg. only when grounded and walking/crouching and maybe to specific hands, etc.

It is also important to note that the procedural hand positioning makes use of post process animation blueprints to enable effective use of twist bones to generate a more realistic visual appearance. Without this, or without appropriate modelled twist bones in your character, you will see distortions of the mesh, particularly at the wrists. You are advised to check the twist bone weightings of your intended characters and the default mannequins can serve as examples here.



Please check out the gameplay demos and see me in discord for any questions.

Support, Feedback and Community.

Discord: https://discord.gg/Pf5Qk6cYb8

Current Release.

v1.7 September 2025.

Latest Release Notes.

Release note details available in discord.

V1.7

- New Realistic Sensor Meshes added to reflect change in emphasis on dual hand sensing.
- Minor bug fix: validity check on TryGetPawnOwner
- A crouch state has been added to the demo scene
- Sensor and Capsule debug bool added to the HandSensingController
- The demo character now has state based replication for movement and actions.
- The function MonitorIKEndPositionForVisuals on the HandSensingController has been rewritten for consistency of behaviour.
- In this release, Procedural shoulder animation is setup as default, so after migration, either setup the shoulders or swap out the animation blueprints.

V1.6

- Minor bug fix
- Experimental: *Procedural Shoulder Adjustment*
- Experimental: *Improved Realism to IK transitions*

V1.5

- Minor bug fixes and quality of life improvements

V1.4

- Bug fixes and quality of life improvements

V1.3

- Conditional Interactions

V1.2

- Replication for Multiplayer.
- Multiple characters can interact with the same object.
- Keypads added to the demo map to demonstrate complex interactions.
- Removed *BP_DT_Cpmt_SensableObjectAction* as no longer used.

V1.1

- Initial bug fixes
- Immediate support for Metahuman hand rotation

Getting Started.

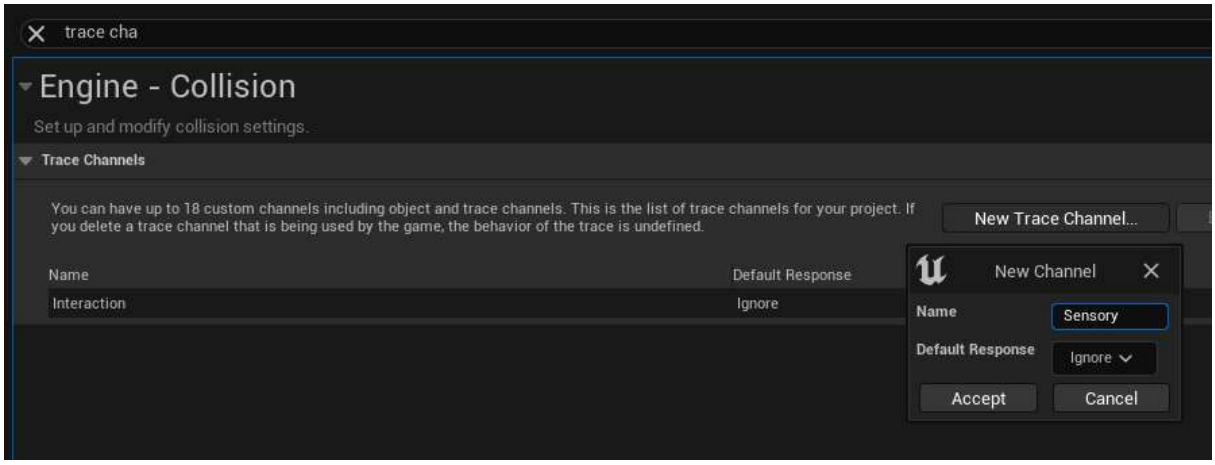
Project Setup.

This introduction to setting up *EnviroSense* in your project, uses the default *Third Person Project* as a basis to illustrate process. You will need to adapt the setup to your own project.

Trace Channels.

Procedural hand positioning uses a “Sensory” Trace Channel:

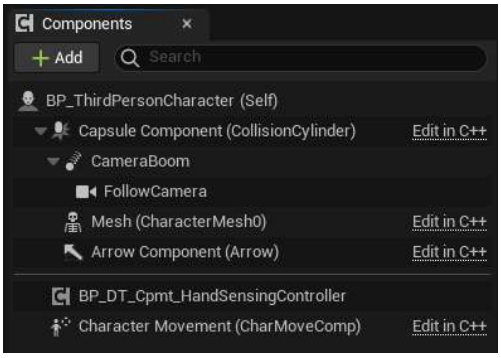
1. In Project Settings, add a new *Trace Channel* called “Sensory” with a *Default Response* of “Ignore”.
2. Objects that use procedural hand positioning will need this set to “Block”.



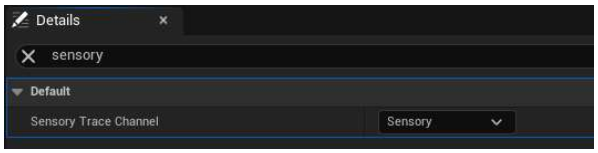
Setting up your character.

Add BP_DT_Cpmt_HandSensingController to your character.

Add the *BP_DT_Cpmt_HandSensingController* to your character:

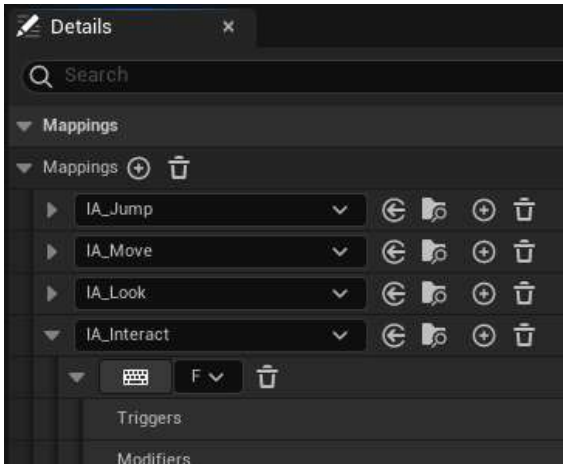


Ensure the Sensory Trace Channel is selected on this component:

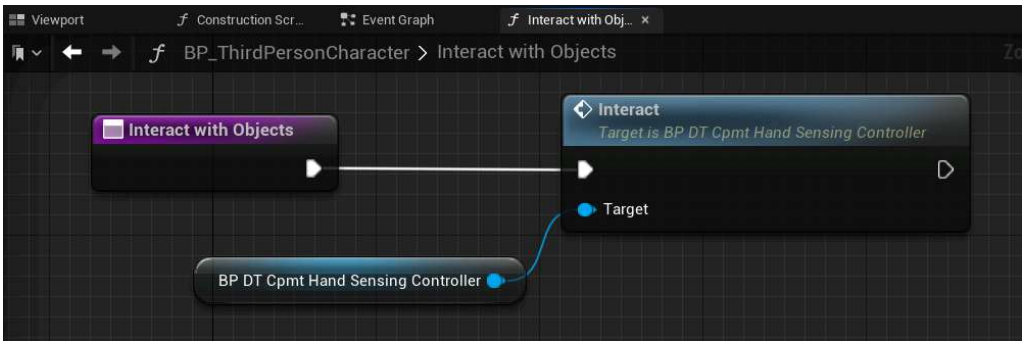


Setting up an input for interactions.

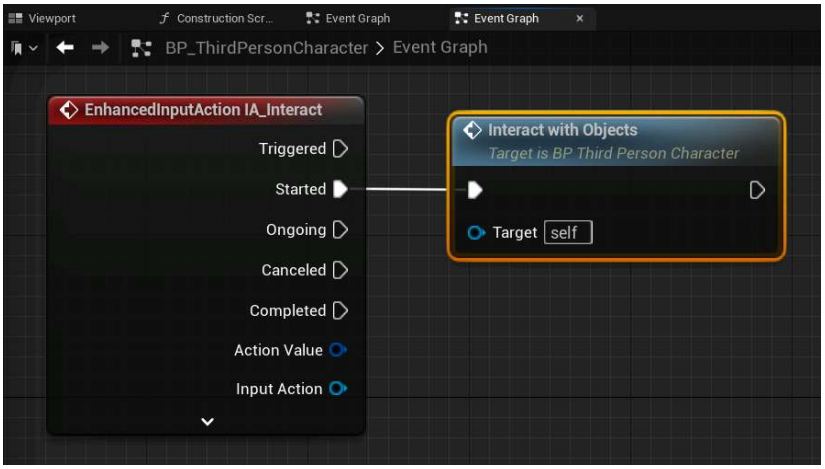
1. Create a new input for ‘Interact’ (example shows *Enhanced Input System*):



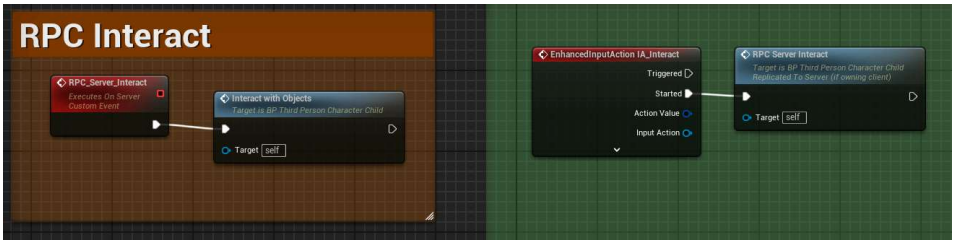
2. Create function in the character controller for input, which references the *BP_DT_Cpmt_HandSensingController*:



3. Link the new ‘Interact with Objects’ function to the Interact input on your character:



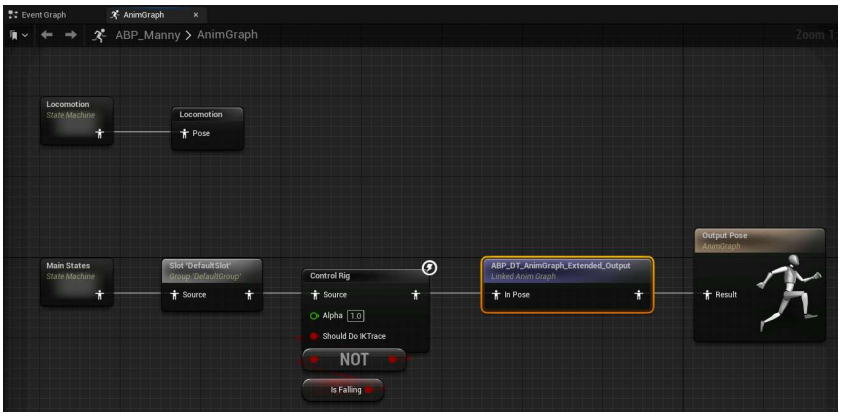
3b. Replicate the interact if required (Set the server event to ‘reliable’):



Adapting the Animation Blueprint.

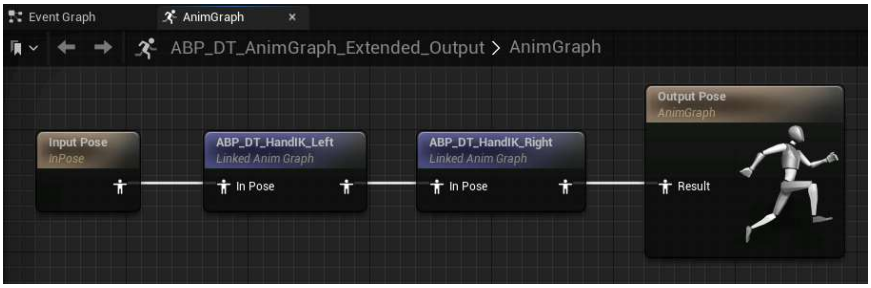
Extending the Animation Blueprint.

1. Add *ABP_DT_AnimGraph_Extended_Output*, if you do not already have a suitable linked *Anim Graph*.



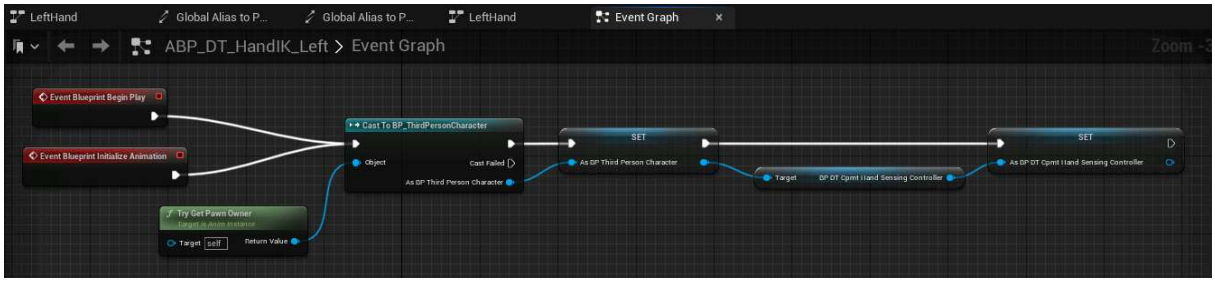
This should be added last in the order so that it is not visually affect by prior animations and, for example, foot IK repositioning. Similarly, since these Anim Graphs blend at the hand, the base animations will be unaffected.

2. Inside the extension are Anim Graphs for each hand (if using your own extension add these).



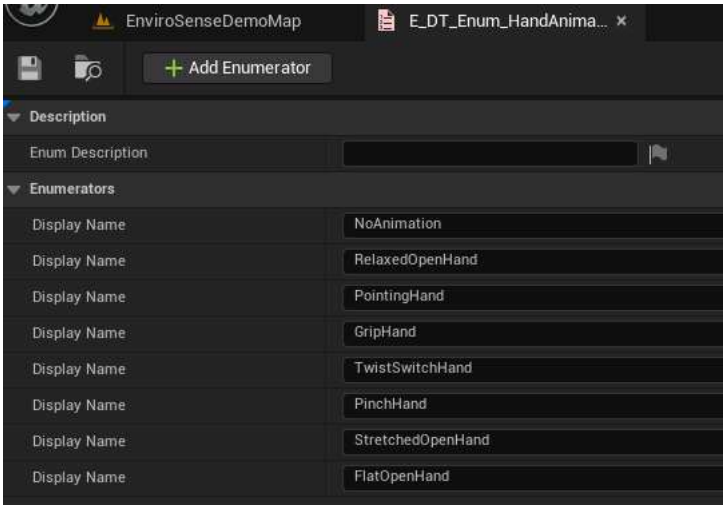
3. The setup of **both** HandIK Anim Graphs, which may require modification to your character setup

NOTE: This is no longer necessary from v1.4 of EnviroSense.

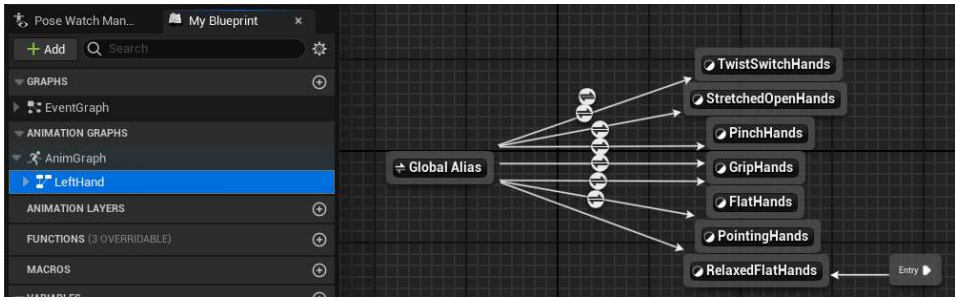


Adding New Hand Animations.

Extend the hand animation enum E_DT_Enum_HandAnimations, by adding a new entry for your new animation.



Add the hand animation to both *ABP_DT_HandIK_Left* and *ABP_DT_HandIK_Right* using the enum category entered above. Entry is from a Global Alias for simplicity:

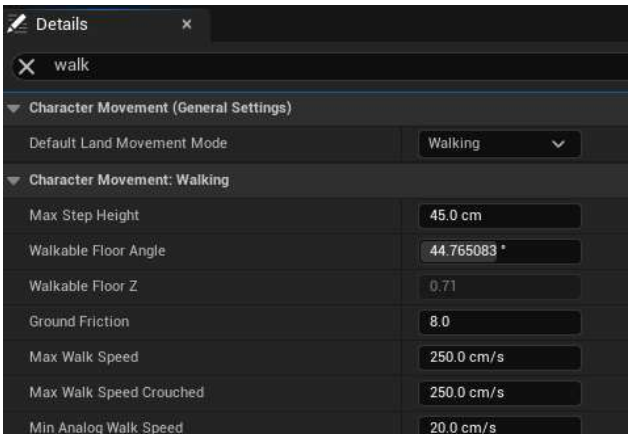


These animations are only blended from the hand so no other aspects of the character animation are affected.

Notes On Character Setup.

Character movement speed and hand sensing.

Using the hand sensing IK at high character movement speeds would be visually poor. You can define the criteria to prevent hand sensing on the controller, but the default behaviour is to only work at low “walking” speeds. To achieve this in the default Third Person Project, just for illustration purposes, we will limit the movement speed “Max Walk Speed” to force a walk:

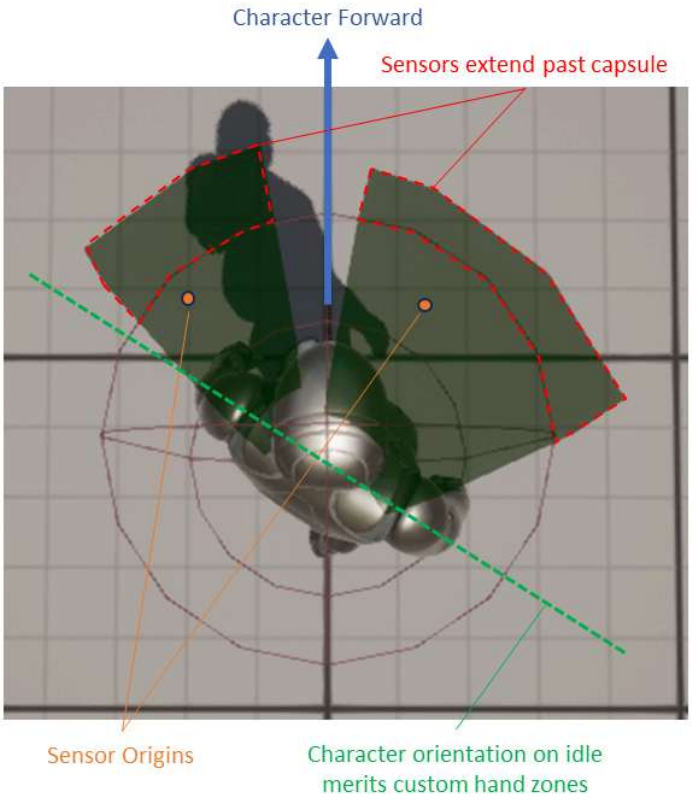


In a properly designed character you may have ‘walk’, ‘jog’, ‘run’ conditions and these can be used to set the criteria, or you can

simply use the speed monitoring that is default in *BP_DT_Cpmt_HandSensingController*..

Sensor Zones:

The character uses sensor zones. The default setup is suite to the default UE5 mannequin and the exact setup should also consider the positioning of the character in an idle stance as that will affect reach. For example, in the default Third Person Project the Idle stance has the character turned ~45° to the right, with left foot forward and this merits sensor zones on the left with limited side range:



A number of colliders are provided for each side (and they can be swapped over) allowing choice for the detection zone that best suits the character and it's idle stance. The colliders should be positioned

and scaled as appropriate to the character, with the following considerations:

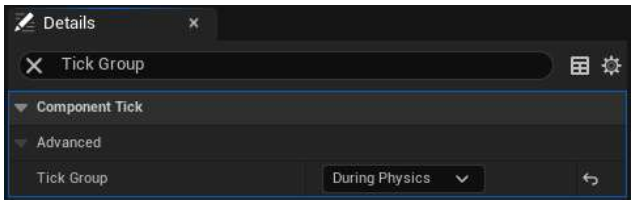
- The maximum range of the collider should account for the reach of the character's wrist (not hand/fingers as you should assume a 90^{o} hand bend may be required).
- The colliders origins should be in front of the player and preferably within the capsule radius (these are used as reference points to select interaction points in front of the player).
- If the colliders reach above the capsule, they should only do so in front of the player to avoid undesirable detections and hand distortions.
- The sensor colliders will also need to be moved as appropriate, e.g. lowered when the character crouches. Sample event triggered methods are included to link up an existing crouch system.
- New collider meshes were provided in v1.7 which facilitate two-handed interactions with objects directly in front of the character. For dynamic sensing, you should avoid placing objects with sharp outprojections for dynamic sensing to prevent hand overlaps (i.e. limit such objects to one hand only).

The sensor colliders are spawned on the character and anchored to the root, whilst this may initially be thought counterintuitive, this is to avoid movement of the colliders in response to the animation of the character which may cause items to enter and leave the zone of influence with undesirable visual effects of activating and cancelling IK.

Any shape of collider can be used to control how the character interacts with its environment and unusual or large colliders will result in undesirable effects.

Delayed IK Movements?

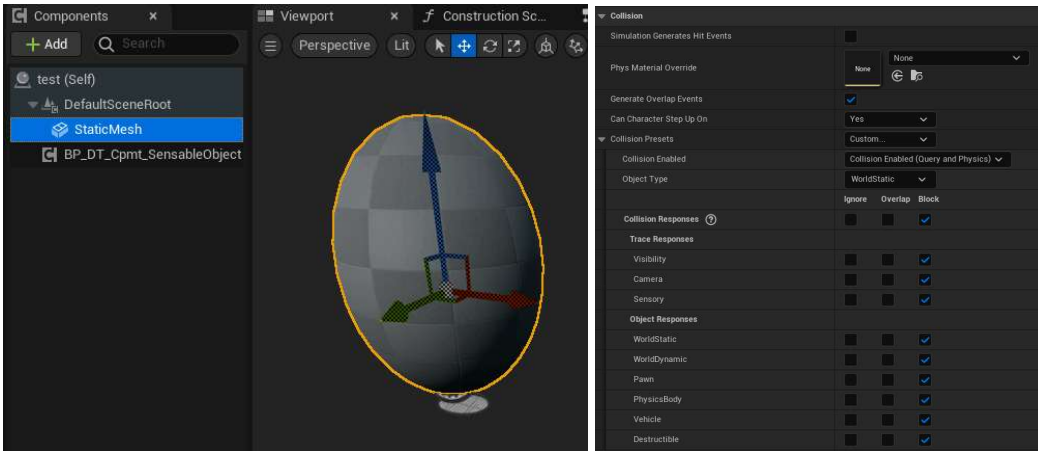
If you observe delays in the IK movement, which may be most evident when interact with precise moving objects, select the mesh and delay its tick group, typically it may be set to *Pre-Physics*:



Setting Up New Interactable Objects.

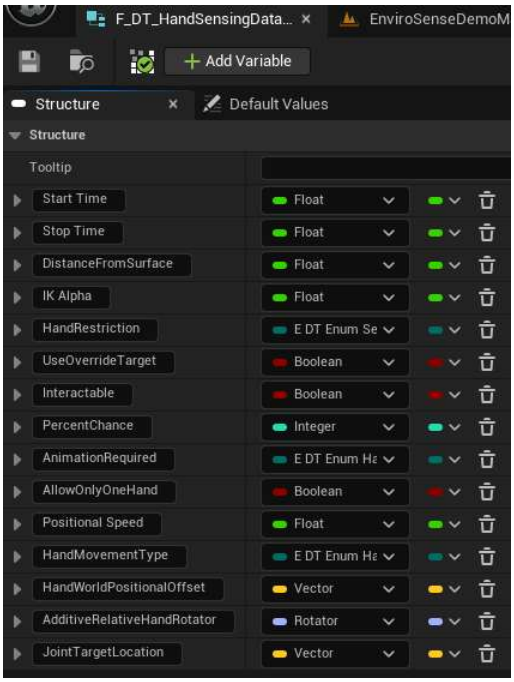
Basic Setup.

1. Create a Blueprint Actor
2. Add a Mesh.
3. If procedural sensing is required set the collision preset to *Custom* and ensure that the *Sensory* trace channel is set to *Block*.
4. Add the *BP_DT_Cpmt_SensableObject* component



Customisation of Sensory Objects.

IK interactions are defined by the *F_DT_HandSensingDataStruct*:



Start Time: Time in seconds to engage a new IK interaction.

Stop Time: Time in seconds to disengage an IK interaction.

Distance From Surface: Distance from the surface that the hand should be placed.

IK Alpha: Strength of the IK interaction – typically set to 1.

Hand Restriction: Is this interaction possible for a specific hand or either?

Use Override Target: Does the Object have specific transforms for precise hand positioning (sensing will no longer be procedural)?

Interactable: Once IK is engaged, can the object be interacted with?

Percent Change: The chance the encounter will result in an IK interaction, typically set to 100, setting a lower number will create an occasional interaction event and make the gameplay less predictable.

Animation Required: Select the specific hand animation, if any, that is required.

Allow Only One Hand: To prevent both hands interacting with an object at the same time. This is advised for precise interactions, such as buttons, but may also make incidental procedural interactions more realistic.

Positional Speed: A transition value that should range between >0 – 1, where 1 will result in instant hand positioning, whereas lower numbers, e.g. 0.2-0.3 will allow the hand to transition smoothly between target locations. This may be particularly useful when multiple colliders are adjacent and being transitioned between

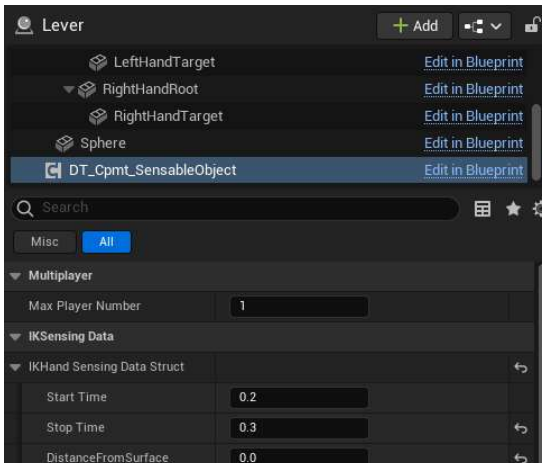
Hand Movement Type: For procedural movement, *Smooth* allows the hand to smoothly transition between targets, gliding over the surface, whereas *Walking* enables hand movement over the surface resulting in a lifting and relocation movement.

Hand World Positional Offset: This should be used very sparingly, it provides a world offset to add variety to procedural interactions, but can create undesirable visual outcomes.

Additive Relative Hand Rotator: Typically used to add a rotation to the hand to ensure it places flat on a surface (or as required) enabling compensation for different rig setups as well as different visual outcomes.

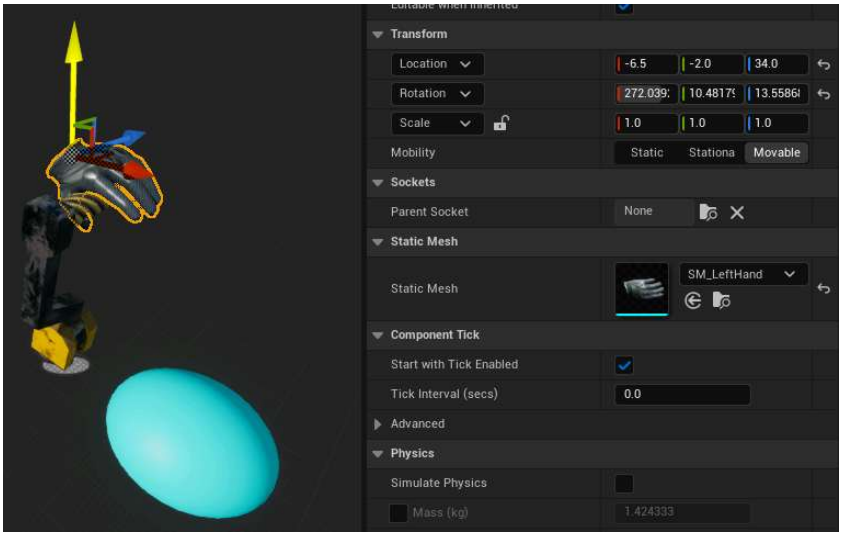
JointTargetLocation: Used as an IK ‘elbow’ hint, in component space. The default value provides good outcomes in mose procedural cases and is automatically flipped depending on the hand.

From v1.1, you now specify the *Max Player Number*, to limit the number of characters that can interact with any object:



Positioning Hand Target Overrides on Specific Objects.

Hand meshes are supplied to facilitate the positioning of the hand targets. Add a static mesh as a hand target. Set the static mesh to eith *SM_LeftHand* or *SM_RightHand* as appropriate to display the hand of Quinn in an approximately correct position and orient the static mesh to the correct positioning. Once close to the correct position, clear the static mesh of the hand mesh and fine tune for your character at runtime.



Blueprint Interfaces.

Several Blueprint Interfaces are used to enable the *BP_DT_SensableObject* component to communicate with the customisations of the specific object blueprint, without modification of the *BP_DT_SensableObject* component:

BI_DT_Sensing_GetHandTargetsAndRoots: Typically used during initiation to identify specific hand targets.

BI_DT_Sensing_BeingDetected: When the character is aware of an object, but a specific IK interaction is yet to fully begin. An example use of this interface is to initiate positioning of hand targets prior to completion of hand positioning to ensure the hand transitions to the correct place.

BI_DT_Sensing_BeingSensed: When an object is being sensed (ie. is the target of Hand IK). Is being sensed is used after a time delay of detection that matches the start time of the objects sensing data. In effect, Being Sensed is called when the hand reaches it's intended position (that the hand is still detecting is checked before calling). A typical use of this is to confirm the hand is in position before allowing an interaction (as seen in the example switches in the demo map).

BI_DT_Sensing_Interact: Used to initiate specific code for an ‘interaction’, for example, operation of a switch. As it’s name implies, this is typically used when an object is interactable to trigger suitable events.

BI_DT_SensingAbortIK: Can be used to abort an IK event when specific atypical criteria are met. Typically used when you wish to abort an IK event, but the standard conditions are not appropriate. An example is seen in the example handrail in the demo map.

Setting Up Different Skeletons.

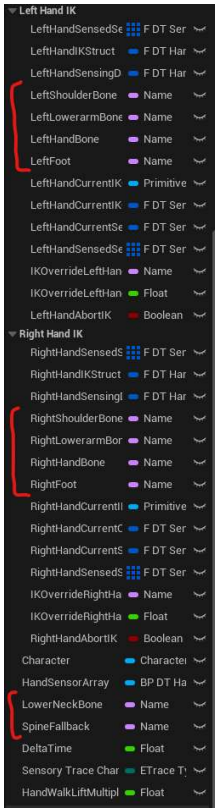
Bone names.

A number of bone name variables exist on the *BP_DT_Cpmt_HandSensingController* they default to the UE5 Mannequin skeleton bone names.

If using the UE4 Mannequin skeleton, you will need to change the name of *SpineFallback* from *spine_05* to *spine_03*.

No changes are required here for Metahumans (although you will need to make the standard changes to the root bone name in the animation blueprint).

If you are using a custom skeleton, you will need to review the bone names appropriately.

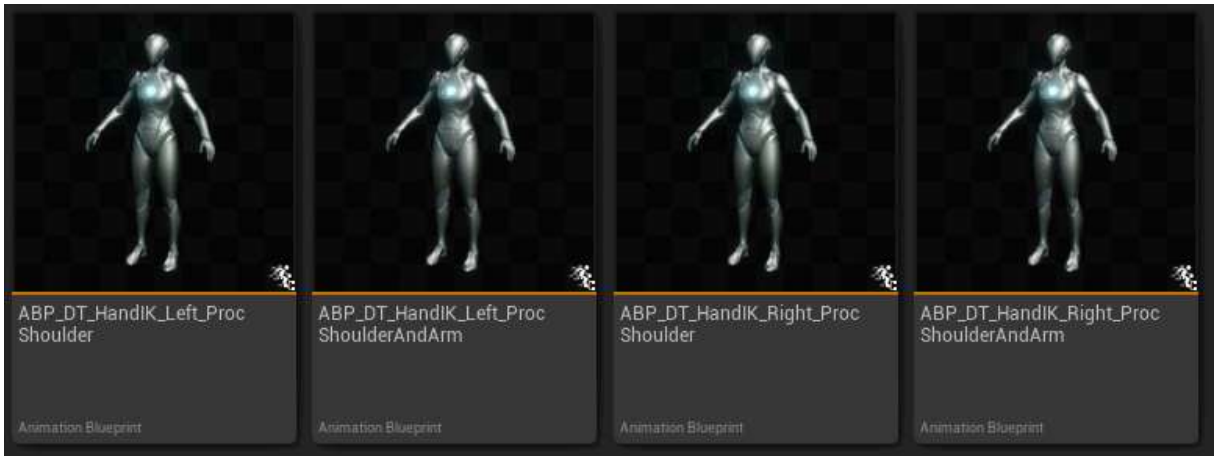


Metahumans and other custom skeletons.

A new vector variable (*RigSpecificHandAdjustment*) has been added to *BP_DT_Cpmt_HandSensingController* to enable corrections for different hand rotations. UE4 and UE5 rigs require not change to the default setting (1,1,1). For example, for metahumans using the metahuman skeleton, and not the the UE5 skeleton, this should be set to (-1,1,-1).

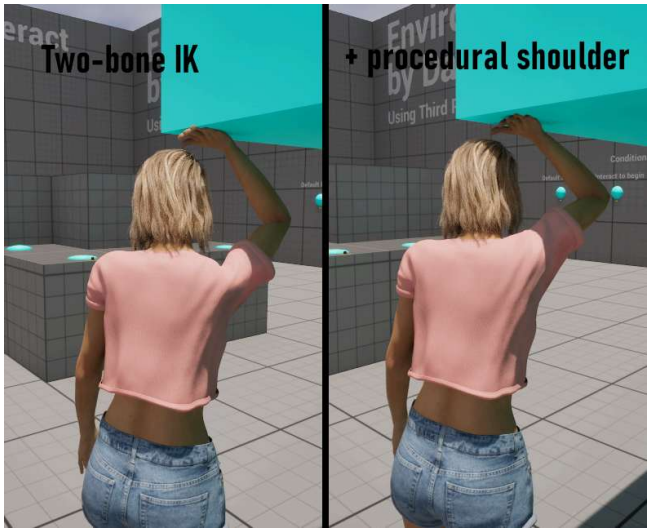
Procedural Shoulder adjustment (v1.6+).

Version 1.6 included 4 new animation blueprints to facilitate procedural shoulder animation:



Procedural Shoulder Adjustment.

Intended outcome:

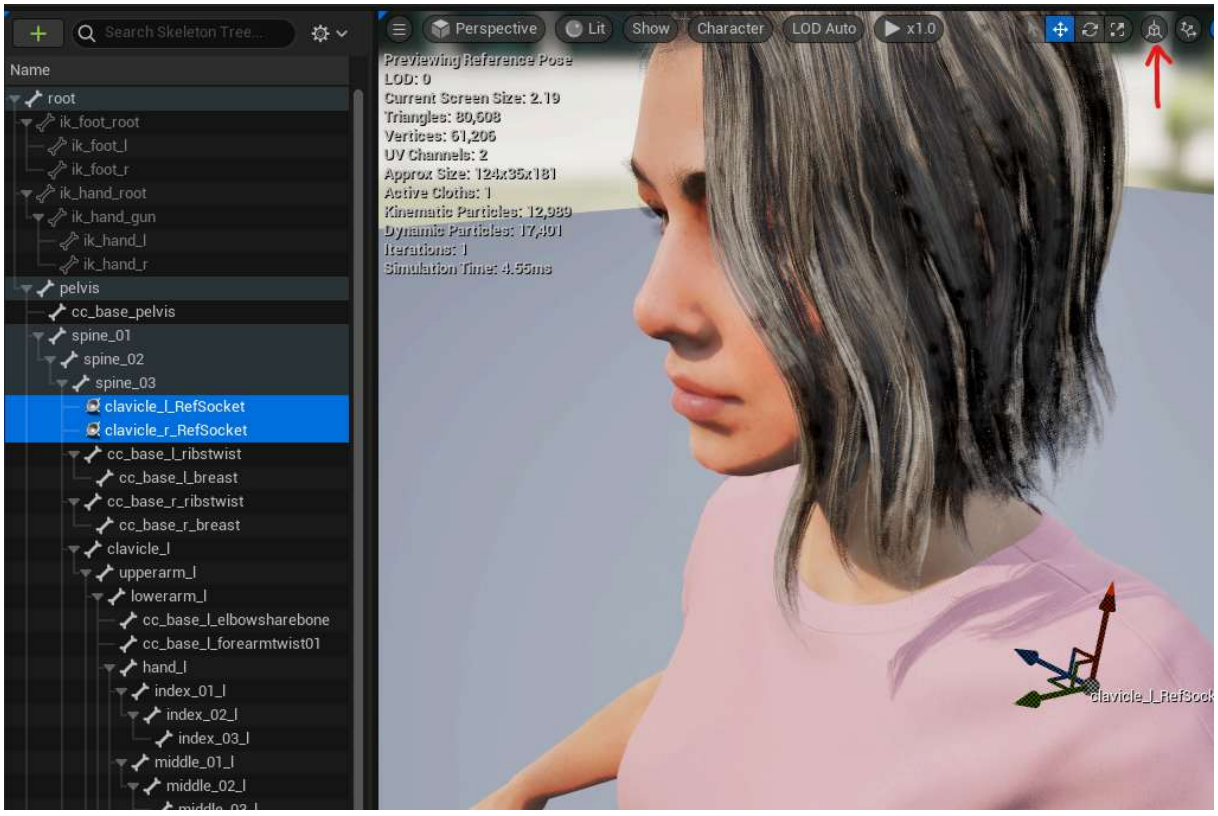


Two replacement for the hand IK animation blueprints have been included that enable procedural adjustment of shoulders. This is particularly useful for more simple skeletons and setups that do not have animation blueprint post-processing.

There is also a float curve provided, which provides an accessible method to fine tune the amount of shoulder lift/drop:

Implementation.

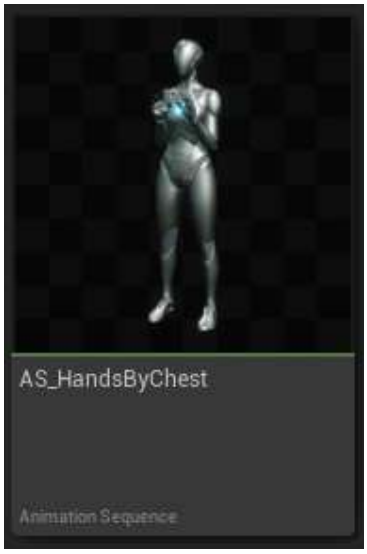
1. Add shoulder reference sockets to the highest chest bone. These will report 'unadjusted shoulder position and orientation based on chest orientation'. Make sure the two names of these sockets match the names used in the 'ClavicleReference' name variables in the Hand IK linked ABPs.



2. Swap the HandIK animation blueprints for the new ProcShoulder variants.

Added realism through blended animation (includes procedural shoulder).

This method makes use of an additional ‘Hands in safe location’ animation to blend when possible the HandIK transitions:



Implementation.

1. Follow the implementation for Procedural Shoulder above, but swap the hand IK animation blueprints for the “ProcShoulderAndArm” variants.